

Guided Lab — Build a Recursive E-mail Harvester in Bash

Day 2 · scripting track · ~90 min · you build one working shell script, line by line

You will build `harvest.sh`: give it a URL, it downloads the page with `curl`, extracts every e-mail address with a regular expression, follows the links on the page, and repeats — until it has crawled a whole site and written a clean `url,email` CSV.

How to read this lab. Every step first asks you to *predict* the line of code. Cover the **Answer** with your hand, write your own version, then reveal. You learn the tool by guessing it, not by copying it.

Rules of engagement. Run this only against the training sandbox below, or a site you are explicitly authorised to test. Harvesting e-mails from real sites is regulated (GDPR). Respect `robots.txt`, throttle your requests, and never republish what you collect.

Step 0 — The sandbox (5 min)

We need a small site we own, so results are predictable and nobody else is touched. Create it and serve it locally:

```
mkdir -p ~/harvest-lab/site/dept && cd ~/harvest-lab/site
cat > index.html <<'EOF'
<html><body><h1>ACME (demo)</h1>
<p>info@acme-demo.test - sales@acme-demo.test</p>
<a href="team.html">Team</a> | <a href="dept/hr.html">HR</a> | <a
href="/offices.html">Offices</a>
<a href="https://external.example.org/z.html">partner</a>
<a href="mailto:webmaster@acme-demo.test">write</a></body></html>
EOF
echo '<html><body>CEO j.doe@acme-demo.test, CTO a.smith@acme-demo.test <a
href="index.html">home</a></body></html>' > team.html
echo '<html><body>Paris paris@acme-demo.test <a href="team.html">team</a></body></html>'
> offices.html
echo '<html><body>Jobs careers@acme-demo.test recruiting@acme-demo.test <a
href="../offices.html">offices</a></body></html>' > dept/hr.html

python3 -m http.server 8099                # leave this running; open a second terminal for
the rest
```

The site now lives at `http://127.0.0.1:8099/`. There are **8 distinct e-mails** spread over 4 pages, plus one `mailto:` link and one **external** link we must *not* follow.

Step 1 — Fetch one page (10 min)

1a. Predict. Which command downloads a page and prints its HTML to the terminal, *without* the progress bar?

Answer `bash curl -s http://127.0.0.1:8099/` `-s` is *silent*: no progress meter, just the body.

1b. Predict. Add three options a serious collector always wants: follow redirects, decompress gzip, and never hang forever.

Answer `bash curl -sL --compressed --max-time 15 http://127.0.0.1:8099/` `-L` follows 301/302, `--compressed` accepts gzip and decodes it, `--max-time 15` caps the whole transfer at 15 s.

1c. Sites often behave differently for "bots". Give `curl` a realistic identity:

```
curl -sL --compressed --max-time 15 -A "Mozilla/5.0 (OSINT training)"
http://127.0.0.1:8099/
```

`-A` sets the `User-Agent` header. You now hold the raw HTML. Next we mine it.

Step 2 — Extract e-mails with a regex (15 min)

An e-mail is roughly: *some allowed characters*, then `@`, then a *domain*, then a dot and a *TLD*.

2a. Predict. Complete this regular expression:

```
_____@_____\._____
```

Answer `[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}` - `[A-Za-z0-9._%+-]+` — the local part (before `@`): letters, digits, dot, underscore, percent, plus, minus. - `@` — the literal separator. - `[A-`

`Za-z0-9.-]+` — the domain labels. `\.[A-Za-z]{2,}` — a dot then a TLD of at least 2 letters. The `\.` is an *escaped* dot (a bare `.` means "any character").

2b. Predict. Which `grep` prints only the matching e-mails (not the whole line), using extended regex?

Answer `bash curl -s http://127.0.0.1:8099/ | grep -oE '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' -o` prints **only** the matched text; `-E` enables **Extended** regex (so `+` and `{2,}` work without backslashes).

2c. The page has `info@` twice (once in text, once in the `mailto:`). Remove duplicates:

```
curl -s http://127.0.0.1:8099/ | grep -oE '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' | sort -u
```

You should now see `info@`, `sales@`, `webmaster@` — the three e-mails on the front page. **That is the whole idea of the original one-liner.** Everything after this is turning it into a real tool.

Step 3 — A first script with a clean CSV (15 min)

Time to save this as `harvest.sh` and make it produce `url,email` rows.

3a. Predict. How do you (1) refuse to run with no argument, and (2) capture the first argument into a variable?

Answer `bash [ -z "$1" ] && { echo "usage: $0 <url>"; exit 1; } URL="$1"` `-z` is true when the string is empty. `$0` is the script's own name.

3b. Write the CSV header, then append one `URL,email` line per address:

```
#!/usr/bin/env bash
set -uo pipefail
URL="$1"; OUT="emails.csv"
echo "url,email" > "$OUT"
curl -sL --compressed --max-time 15 -A "Mozilla/5.0 (OSINT training)" "$URL" \
  | grep -oE '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' | sort -u \
  | while IFS= read -r MAIL; do echo "$URL,$MAIL" >> "$OUT"; done
column -s, -t "$OUT"
```

Run it:

```
chmod +x harvest.sh
./harvest.sh http://127.0.0.1:8099/
```

Checkpoint: three rows, all from the front page. `set -uo pipefail` makes the script stop on an undefined variable or a failed pipe — a habit worth keeping.

Step 4 — Follow the links (the real work, 25 min)

One page is a scraper. A **crawler** discovers new pages by reading the links on the pages it already has. That means a loop and a to-do list.

4a. Extract the links

Predict. Which pipeline pulls every `href="..."` target out of the HTML?

Answer `bash grep -oE 'href="[^"]+"' | sed -E 's/^href="//; s/"$//'` `grep -oE 'href="[^"]+"'` grabs `href="..."`; the `sed` strips the `href="` prefix and the trailing quote, leaving the bare URL.

4b. Does curl recurse on its own?

A student asked in class: *"Isn't there a curl option that follows links automatically?"*

No. `curl` fetches exactly the URLs you give it — there is no recursive-download flag. The tool that *does* have built-in recursion is `wget`:

```
wget -r -l 2 -np http://127.0.0.1:8099/      # -r recurse, -l depth, -np no parent
```

But `wget -r` just downloads files; it gives you no hook to run your regex per page, no domain filter you control, no CSV. So for harvesting we **build the recursion ourselves** around `curl`. That is the real skill, and it is the same pattern every crawler uses.

4c. The crawl loop

We keep two files: a **queue** of URLs still to visit (each tagged with its depth), and a **visited** list so we never fetch the same URL twice.

Predict. How do you mark a URL as visited and skip it if it already is?

Answer `bash grep -qxF "$URL" "$VISITED" && continue # already seen -> skip echo "$URL"`
`>> "$VISITED" # otherwise record it -q quiet, -x whole-line match, -F fixed string (no regex surprises from dots in URLs).`

4d. Resolve the links you find

Links come in four shapes. Predict the branch for each before revealing.

Link in the page	Must become
<code>mailto: / javascript: / #...</code>	<i>skipped</i>
<code>https://other-domain/...</code>	<i>skipped</i> (off-site)
<code>/offices.html</code> (absolute path)	<code>http://<domain>/offices.html</code>
<code>team.html</code> or <code>../offices.html</code> (relative)	resolved against the current page's folder

Answer — a case on the link handles all four: `bash case "$LINK" in mailto:*|javascript:*|"#"*) continue ;; http://*|https://*) ["$(domain "$LINK")" = "$BASE_DOMAIN"] || continue ;; /*) LINK="http://${BASE_DOMAIN}${LINK}" ;; *) # relative: climb one folder per leading ../ BASE="$CURDIR"; REL="$LINK" while ["${REL#../}" != "$REL"]; do REL="${REL#../}"; BASE="${BASE%/}"; BASE="${BASE%*/}"; done LINK="${BASE}${REL}" ;; esac` The relative case is the subtle one: `../offices.html` seen from `.../dept/hr.html` must climb out of `dept/`. Each `../` drops one folder from the current directory.

Step 5 — The finished script (10 min)

Put it together. This is the complete, working `harvest.sh` — read every line and match it to the step it came from.

```
#!/usr/bin/env bash
# harvest.sh - recursively crawl one website and collect e-mail addresses.
# Usage: ./harvest.sh <start-url> [max-depth]          (default depth 2)
set -uo pipefail

START="${1:-}"; MAX_DEPTH="${2:-2}"; OUT="emails.csv"
UA="Mozilla/5.0 (OSINT training; +harvest.sh)"
EMAIL_RE='[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}'
[ -z "$START" ] && { echo "usage: $0 <url> [max_depth]"; exit 1; }
```

```

BASE_DOMAIN="$(echo "$START" | sed -E 's#^https?:/##; s#/.*##')"
WORK="$(mktemp -d)"; VISITED="$WORK/visited"; QUEUE="$WORK/queue"
: > "$VISITED"; echo "$START|0" > "$QUEUE"; echo "url,email" > "$OUT"

while [ -s "$QUEUE" ]; do
    ENTRY="$(head -n1 "$QUEUE")"; sed -i.bak '1d' "$QUEUE" && rm -f "$QUEUE.bak"
    URL="{ENTRY%|*}"; DEPTH="{ENTRY##*|}"
    grep -qxF "$URL" "$VISITED" && continue
    echo "$URL" >> "$VISITED"

    HTML="$(curl -sL --max-time 15 --compressed -A "$UA" "$URL")" || continue
    echo "[depth $DEPTH] $URL"
    echo "$HTML" | grep -oE "$EMAIL_RE" | sort -u | while IFS= read -r MAIL; do
        echo "$URL,$MAIL" >> "$OUT"
    done

    [ "$DEPTH" -ge "$MAX_DEPTH" ] && continue
    CURDIR="{URL%/*}/"
    echo "$HTML" | grep -oE 'href="[^\"]+"' | sed -E 's/^href="//; s/"$//' | sort -u |
while IFS= read -r LINK; do
    case "$LINK" in
        mailto:*|javascript:*|"#"*) continue ;;
        http://*|https://*) [ "$(echo "$LINK" | sed -E 's#^https?:/##; s#/.*##')" =
"$BASE_DOMAIN" ] || continue ;;
        /*) LINK="http://{BASE_DOMAIN}${LINK}" ;;
        *) BASE="$CURDIR"; REL="$LINK"
            while [ "${REL#./}" != "$REL" ]; do REL="${REL#./}"; done
            while [ "${REL#..}" != "$REL" ]; do REL="${REL#..}"; BASE="${BASE%/}";
BASE="${BASE%/*}/"; done
            LINK="{BASE}${REL}" ;;
    esac
    echo "${LINK}|${((DEPTH+1))}" >> "$QUEUE"
done
done

PAGES="$(sort -u "$VISITED" | wc -l | tr -d ' ')"
MAILS="$(tail -n +2 "$OUT" | cut -d, -f2 | sort -u | wc -l | tr -d ' ')"
rm -rf "$WORK"
echo "---"; echo "pages crawled: $PAGES    unique emails: $MAILS"

```

Run it across the whole sandbox:

```

./harvest.sh http://127.0.0.1:8099/ 3
column -s, -t emails.csv

```

Expected result: it visits the front page and the three linked pages, skips the external partner link and the `mailto:`, and reports **8 unique emails**. If you get that, your recursive harvester works.

Step 6 — Read the queue mechanics (5 min)

Answer these from your own script. They prove you understand the recursion, not just that it runs.

6.1 Each queue entry looks like `http://.../team.html|1`. What does the number mean, and which two lines split it back apart?

6.2 Change the depth argument from `3` to `0`, then `1`, then `2`. Explain the number of pages crawled each time.

6.3 Remove the `grep -qxF "$URL" "$VISITED"` guard. What happens on this sandbox (`team.html` links to `index.html` which links back)? Why is the visited-set essential?

6.4 Why do we filter on `BASE_DOMAIN`? What would `./harvest.sh https://en.wikipedia.org/ 3` do without it?

Optional Extensions (Bonus)

- **Politeness.** Add a `sleep 1` between fetches and read `robots.txt` first (`curl -s "$BASE/robots.txt"`). A crawler that ignores throttling is both rude and easy to block.
- **De-obfuscation.** Real pages hide addresses as `name [at] domain [dot] com`. Add a `sed` pass that rewrites `[at]` → `@` and `[dot]` → `.` before the regex.
- **Canonical URLs.** `http://host/` and `http://host/index.html` are the same page but crawled twice. Normalise them so each page is fetched once.
- **Richer selectors.** Extend the script to also collect phone numbers, or `mailto:` targets that never appear in the visible text.
- **Point it at authorised infrastructure.** With written permission, run it against a scoped target and compare against `theHarvester` (`theHarvester -d target.tld -b all`). Where does your script win, where does the mature tool win?